

Yale Project

Student name: *Daniele Mensorio*

Course: *Numerical methods for Data Analysis* – Professor: *Prof.essa Rosanna Campagna*
Due date: *April 9th, 2024*

Yale Face Database is a popular facial image database, containing 165 gray-scale images in GIF format of 15 individuals. There are 11 images per subject, one per different facial expression or configuration: center-light, w/glasses, happy, left-light, w/no glasses, normal, right-light, sad, sleepy, surprised, and wink.

Yale64x64.mat file contains two data matrix: 'fea' and 'gnd'.

1. Each row of **fea** is a face that has 4096 pixels - columns - (64×64); $fea \in \mathbb{R}^{165 \times 4096}$
2. **gnd** is the label matrix that associate to each row the individual to which the pic in the 'fea' matrix belongs. $gnd \in \mathbb{R}^{165 \times 1}$

Part a

Let's visualize all the 11 faces of the 15 individuals.

Fea Matrix

11 different faces for 15 different individuals

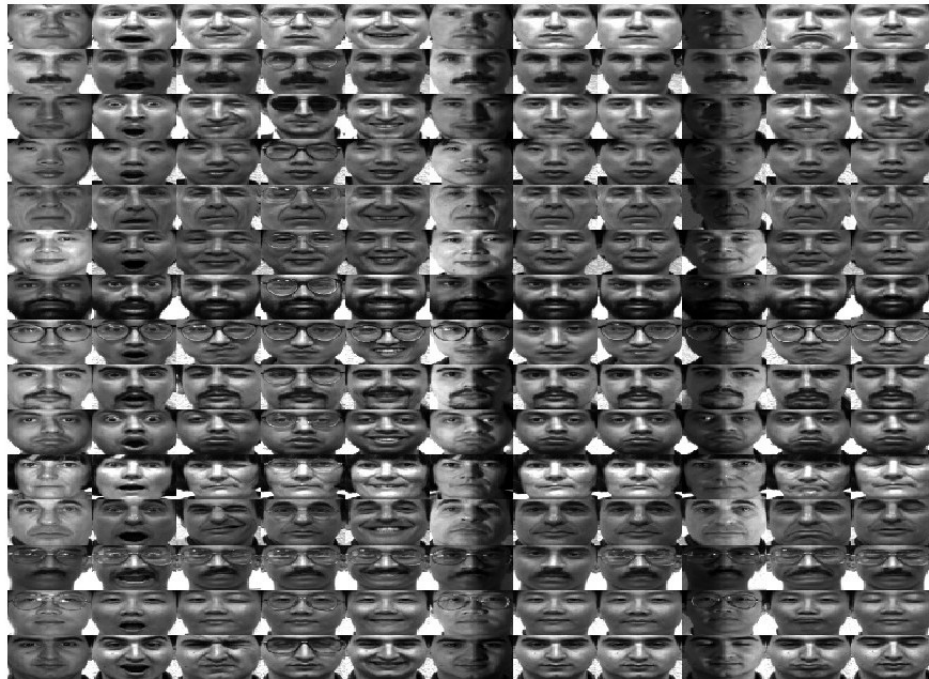


Figure 1: Gray-scale image that contains all the rows of the *fea* data matrix reshaping in a square 64×64 pixel that has all the 11 faces of each 15 individual.

Now, Now, let's create a sub-matrix F in which we store only 6 different faces (different in such a way it is possible to have a good representation of the whole group of individual) for 5 different individual in a gray-scale image, as can be seen in *figure 2*.

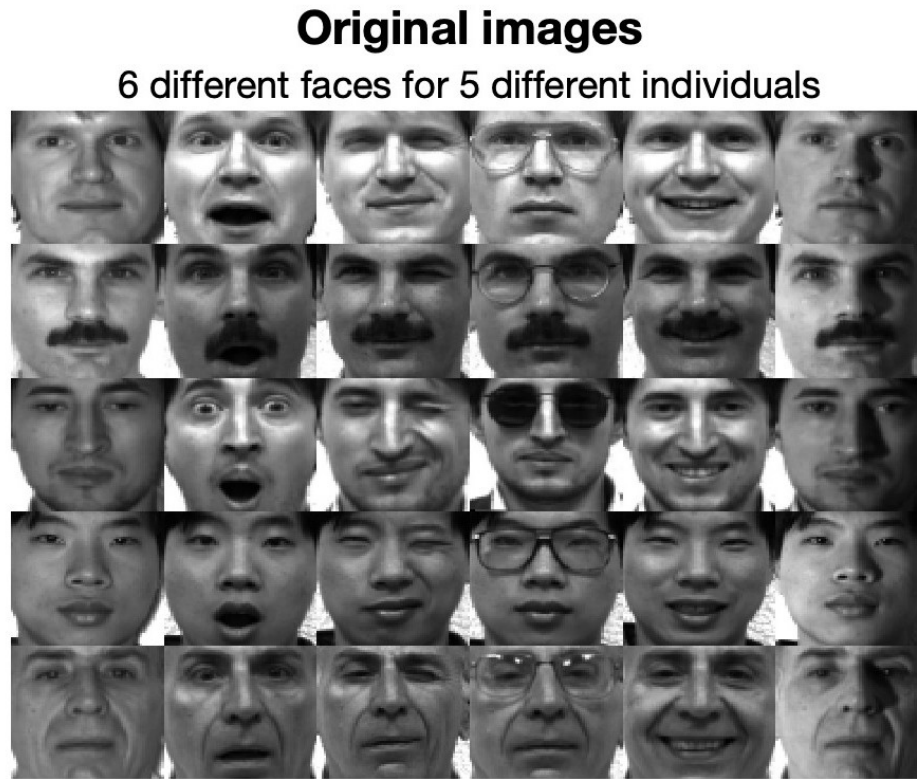


Figure 2: Gray-scale image corresponding to the sub-matrix F , that store only 6 different faces for each 5 individual

Part b

I compute the thin SVD (thin Singular Value Decomposition) for $k = 30$ since I take $6 \times 5 = 30$ rows from the original *fea* data matrix:

$$X_k = U_k D_k V_k^T$$

where:

$U_k \in \mathbb{R}^{30 \times 30}$ is a matrix with orthonormal columns that contains the left singular vectors

$D_k \in \mathbb{R}^{30 \times 4096}$ is a diagonal matrix that contains the singular values

$V_k \in \mathbb{R}^{4096 \times 4096}$ is a matrix with orthonormal columns that contain the right singular vectors

Plot the singular values (that corresponds to the diagonal values of the D_k matrix) in a logarithmic scale (*figure 3*) so that it is possible to visualize them better.

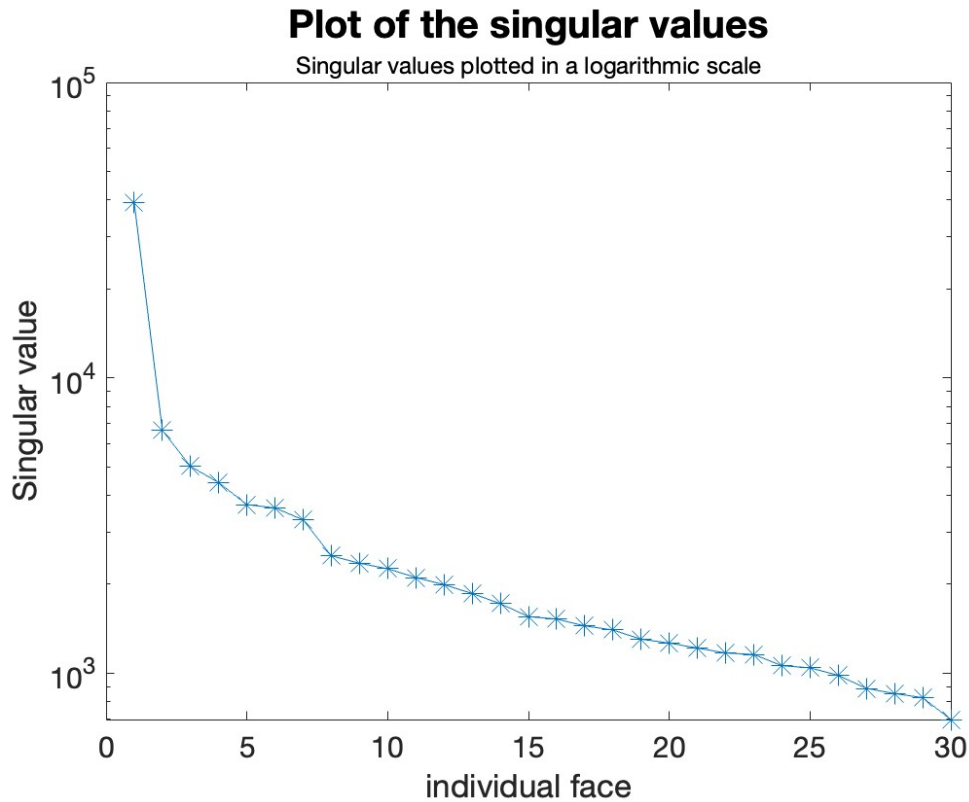


Figure 3: Plot of the singular values σ_j of the diagonal matrix D , result of the thin SVD (thin singular value decomposition) of the data sub-matrix F , in a logarithmic scale.

From *figure 3*, It can be notice that, after a certain number of singular values (in this case, 7) in which the decrease is not very steep, their logarithm decreases rapidly. That could mean that $k = 7$ feature vectors can be taken as a good approximation of the original images, noticing, always, that the best k -approximation happens when $k = \text{rank}(F)$.

Part c

Now, since our matrix is made of 64×64 pixels that made up an image, that correspond to an individual, let's see what can we observe in plotting the first 5 feature vectors (columns of U_k) as gray-scale images.

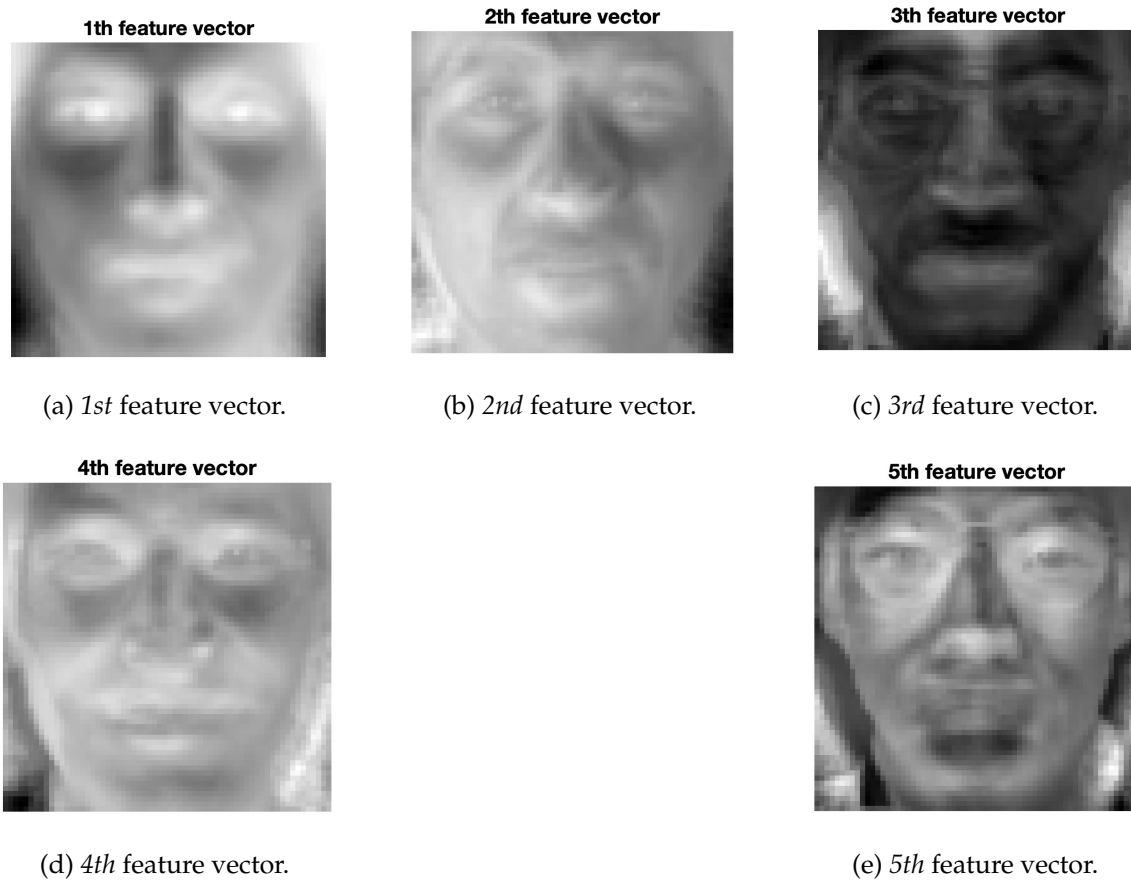


Figure 4: Gray-scale images corresponding to the feature vectors images (64×64 pixels) of the U_k matrix with orthonormal column vectors, obtaining by performing the thin SVD on the sub-matrix F .

From *figure 6*, we can observe that the feature vectors' image capture the main characteristics of the images, contained in the sub-matrix F : in this case, the main feature of the face of these individuals are the eye, the nose and the line of the face, while other not very important characteristic, like the glasses and the eyebrows, cannot see very well in the images.

That prove that as the number of feature vectors increases, more characteristics are visible in the approximation of the images and the pics became more clear.

Part d

Since F dataset contains, still (even if we are working with a sub-matrix, and not with the entire original data), a lot information ($F \in \mathbb{R}^{30 \times 4096}$ or $F \in \mathbb{R}^{320 \times 384}$), the original data can be represented by only a few set of information that allow a big reduction useful both for computational effort and also for eliminate all the noise and capture only the essential information from the column vectors.

Let's compute the first principal components of the data by linear combination of the first k ($k = 4, 8, 15$) feature vectors $u^{(j)}$,

$$x_c^{(j)} = U_k D_k V_k^T = U_k z^{(j)}$$

$$z^{(j)} = D_k V_k^T e_j$$

$$z^{(j)} = u_k^T x^{(j)}$$

$$Z = U_k^T X$$

where $z^{(j)}$ are the principal components of $x^{(j)}$. I plot the approximation as images, as well as the residual,

$$x^{(j)} - P_k x^{(j)}$$

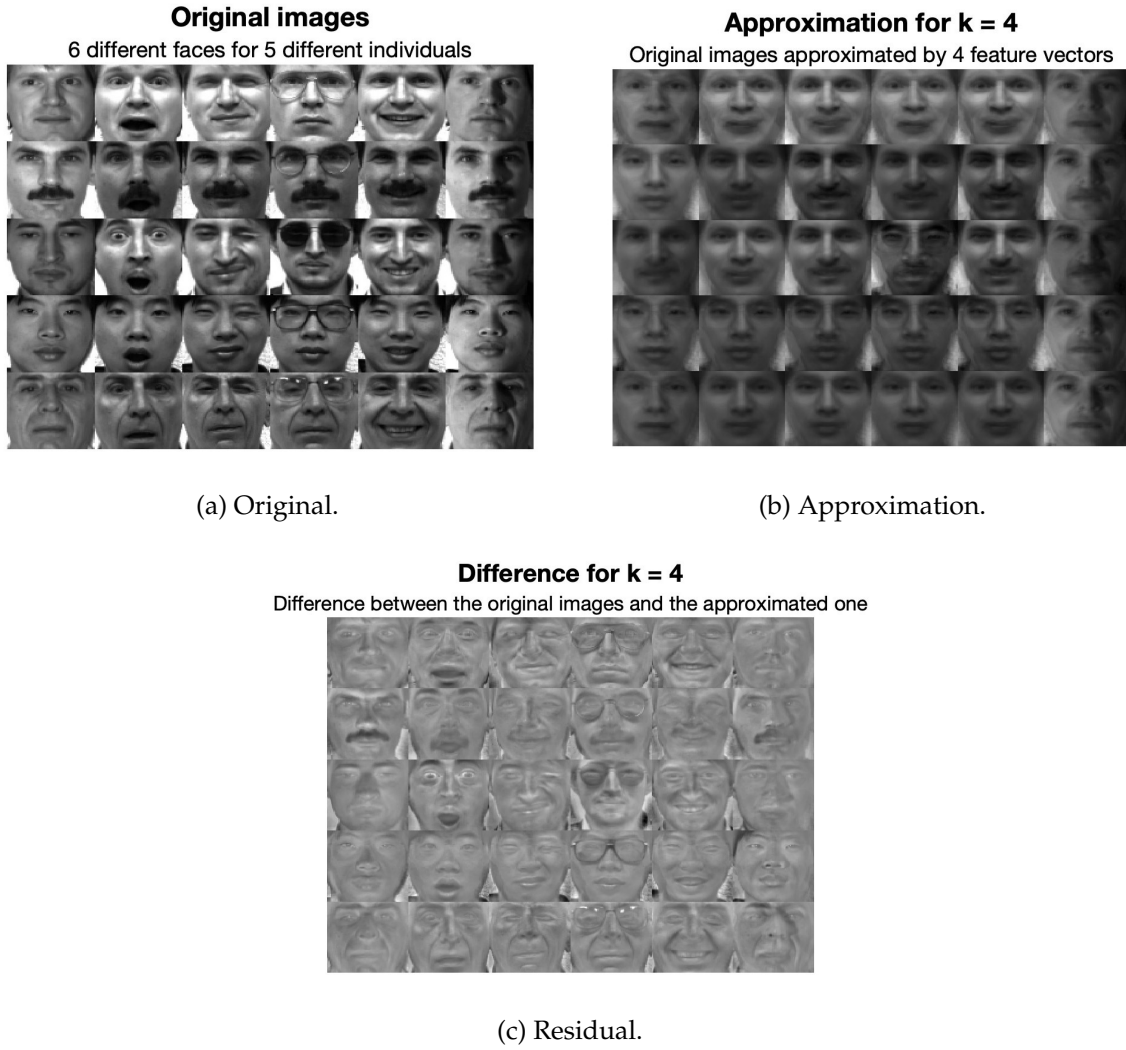


Figure 5: Original data, approximation images and residual between the former and the approximated images by $k = 4$ feature vectors.

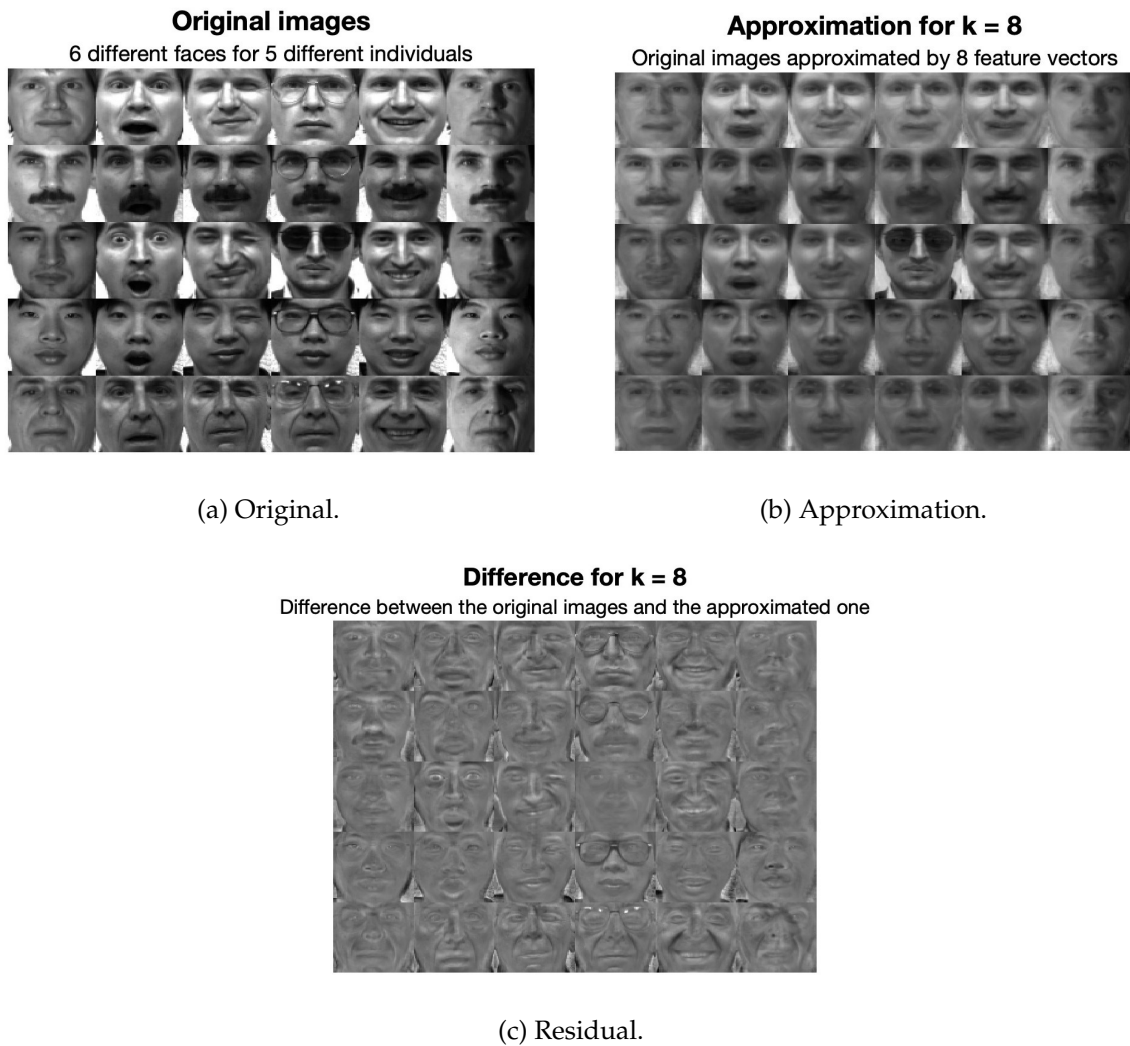


Figure 6: Original data, approximation images and residual between the former and the approximated images by $k = 8$ feature vectors.

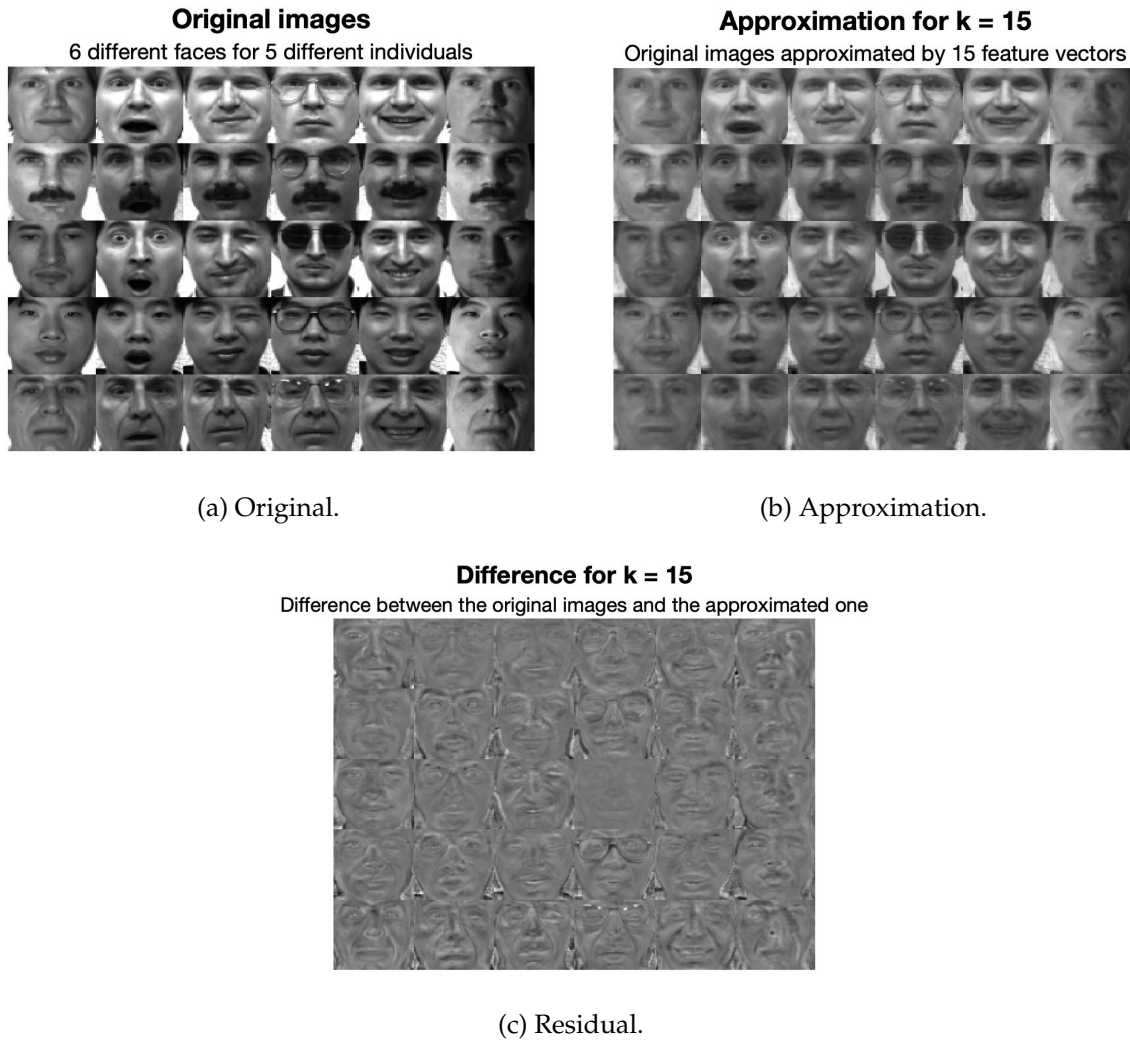


Figure 7: Original data, approximation images and residual between the former and the approximated images by $k = 15$ feature vectors.

from *figure 6* to *figure 7*, it's evident that as the number of feature vectors increase, increase the amount of information (features) that can be seen in the face of the individual but decrease the amount of information that we can capture from the residual (the difference between the original column vectors and the approximated one). That is clear, especially, for $k = 4$ and $k = 15$.

Appendices

```

clear all
load Yale_64x64.mat

faceW = 64;
faceH = 64;
numPerLine = 11;
ShowLine = 15;
Y = zeros(faceH*ShowLine, faceW*numPerLine);
for i=0:ShowLine-1
    for j=0:numPerLine-1
        Y(i*faceH+1:(i+1)*faceH, j*faceW+1:(j+1)*faceW) =
            reshape(fea(i*numPerLine+j+1,:), [faceH, faceW]);
    end
end
figure()
imagesc(Y); colormap(gray);
axis off;
title("Fea Matrix", FontSize=22)
subtitle("11 different faces for 15 different individuals",
    FontSize=18)

% (a) Extract in a submatrix F and plot 6 different faces for 5
%      different
%      individuals as gray scale images. Choose faces that you think
%      are in
%      some sense representative between the different groups (
%      normal, sad,
%      happy, surprised, sleepy, wink).
n_col = 6; %faces
n_row = 5; %individuals
K = zeros(faceH*n_row, faceW*n_col);
for i=0:n_row-1
    for j=0:n_col-1
        K(i*faceH+1:(i+1)*faceH, j*faceW+1:(j+1)*faceW) =
            reshape(fea(i*11+(j+1),:), [faceH, faceW]);
    end
end
figure()
imagesc(K); colormap(gray);
axis off;
title("Original images", FontSize=22)
subtitle("6 different faces for 5 different individuals",
    FontSize=18)

```

```

% (b) After having computed the first singular values/vectors
      of F,
% plot the singular values and comment on how fast (or slowly)
      they
% decrease. Notice, however, that computing the full SVD may be
      very slow,
% so use svds, increasing gradually r. It may be more
      informative to plot
% their logarithms.
n_col = 6; %faces
n_row = 5; %individuals
K_fea = zeros(n_col*n_row, faceH*faceW);
counter = 0;
for i=0:n_row-1
    for j=0:n_col-1
        counter = counter + 1;
        K_fea(counter,:) = fea(i*11+(j+1),:);
    end
end
r = rank(K_fea); %it's equal to 30
[U,D,V] = svd(K_fea);
figure()
semilogy(diag(D), '*-','MarkerSize', 10)
set(gca, 'FontSize', 15)
ylabel("Singular value")
xlabel("individual face")
title("Plot of the singular values", FontSize=20)
subtitle("Singular values plotted in a logarithmic scale",
        FontSize=12)

% (c) Plot the first 5 feature vectors (columns of U) as gray
      scale images.
%in this case, I have that the right singular vectors
for j = 1:5
    figure()
    imagesc(reshape(V(:,j), faceH, faceW))
    colormap(gray);
    axis('equal')
    axis('off')
    title(j + "th feature vector", FontSize=22)
end

% (d) Approximate the 5 images corresponding to the columns
      that you

```

```

% selected by a linear combination of the first k = 4; 8; 15
% feature
% vectors with coefficients their principal components. Each
% time display
% the approximation and difference between it and the original
% data in the
% form of a gray scale image.
n_col = 6;
n_row = 5;
for k = [4, 8, 15]
    Z_k = U(:,1:k)' * K_fea;
    X_k = U(:,1:k) * Z_k;
    fea_diff = K_fea - X_k;
    APPR = zeros(faceH * n_row, faceW * n_col);
    DIFF = zeros(faceH * n_row, faceW * n_col);
    counter = 0;

    for i = 0:n_row - 1
        for j = 0:n_col - 1
            counter = counter + 1;
            % Approximation matrix
            APPR(i*faceH+1:(i+1)*faceH, j*faceW+1:(j+1)*faceW)
                = reshape(X_k(counter, :), [faceH, faceW]);

            % Difference matrix
            DIFF(i*faceH+1:(i+1)*faceH, j*faceW+1:(j+1)*faceW)
                = reshape(fea_diff(counter, :), [faceH, faceW]);
        end
    end
    figure;
    imagesc(APPR);
    colormap gray;
    title("Approximation for k = " + k, FontSize=22);
    subtitle("Original images approximated by " + k + " feature
        vectors", FontSize=18)
    axis off;

    figure;
    imagesc(DIFF);
    colormap gray;
    title("Difference for k = " + k, FontSize=22);
    subtitle("Difference between the original images and the
        approximated one", FontSize=18)
    axis off;
end

```